

Efficient Closed-Form Continuous-Time Neural Networks on Commodity Microcontrollers

Manuel Yobani Martinez Sanchez
OmniTrain Research Group

Abstract—Deploying sequence-modeling neural networks on deeply embedded hardware (TinyML) remains a formidable challenge. The primary bottlenecks are the strict Static Random-Access Memory (SRAM) constraints of microcontrollers and the irregular sampling rates (temporal jitter) inherent to physical sensors. Traditional Recurrent Neural Networks (RNNs), such as LSTM and GRU architectures, assume discrete and synchronous time steps. Furthermore, they are typically deployed via schema-heavy execution frameworks that incur significant dynamic memory overhead.

In this paper, we present an end-to-end systems-engineering pipeline for the native, bare-metal deployment of Closed-form Continuous-time (CfC) neural networks on highly constrained hardware (ESP32-S3). By utilizing a custom, static-memory Zero-Copy binary payload architecture, we bypass traditional framework overhead. Our empirical evaluation against standard LSTM and GRU deployments demonstrates resilience to sensor jitter on a Processor-in-the-Loop (PiL) Inverted Pendulum dataset, mathematical parity with full-precision (FP32) PyTorch models ($MSE < 10^{-6}$), lower inference latency, and a reduced SRAM and energy footprint. All pipelines and the C++ inference engine are open-sourced for reproducibility¹.

Index Terms—TinyML, Continuous-Time Neural Networks, Edge Computing, Zero-Copy Execution, Embedded Systems, Microcontrollers

I. INTRODUCTION

The proliferation of autonomous edge robotics has driven a demand for localized, low-latency artificial intelligence. Processing time-series data locally on microcontrollers (MCUs) ensures deterministic response times, preserves privacy, and reduces dependency on high-bandwidth cloud infrastructure. However, state-of-the-art kinematic sequence models are fundamentally misaligned with embedded hardware.

First, standard discrete-time models, such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), fail mathematically when exposed to irregular sampling intervals. Real-world sensors suffer from packet loss and varying clock frequencies, introducing temporal jitter.

Second, the structural execution of these models relies on generalized inference engines (e.g., TensorFlow Lite for Microcontrollers). These frameworks utilize dynamic memory arenas that consume substantial portions of the restricted SRAM available on MCUs. Furthermore, heavy memory access patterns increase dynamic power consumption.

In this work, we bring Closed-form Continuous-time (CfC) architectures [1] to bare-metal execution environments through

a novel deployment pipeline. Rather than proposing a new neural architecture, our specific systems-engineering contributions are:

- A custom continuous-time ODE solver optimized natively for Xtensa architectures.
- An Execute-in-Place (XIP) Tensor Mapping architecture (.omnibit) that maps Flash memory directly to the CPU cache, bypassing SRAM allocation for network parameters.
- Extensive benchmarking proving that CfCs execute faster, consume less energy, and provide superior robustness to sensor degradation compared to equivalent LSTMs and GRUs.

II. BACKGROUND AND RELATED WORK

A. Continuous-Time Neural Networks

Inspired by the nervous system of the *C. elegans* nematode, Liquid Time-Constant (LTC) networks [2] introduced continuous-time dynamics to artificial recurrent networks. More recently, Hasani et al. developed a closed-form approximation (CfC) [1]. Despite their theoretical efficiency, their practical execution characteristics on deeply embedded hardware have remained underexplored.

B. Embedded ML and Memory Bottlenecks

TinyML research [5] largely focuses on reducing model size through Quantization (e.g., INT8). While highly effective at reducing Flash requirements, quantization often destroys the dynamic range required for differential equations. Furthermore, our architecture avoids generalized inference engines entirely. A critical limitation of frameworks like TFLite Micro [3] is the absence of native ODE solver primitives. Attempting to deploy continuous-time architectures on such frameworks forces the mathematical kernel to be decomposed into discrete scalar operations, resulting in catastrophic computational overhead and an explosion in dynamic memory allocation [4]. Our approach implements a static memory map to achieve Execute-in-Place (XIP) parameter mapping, drastically reducing the active SRAM footprint. This memory efficiency allows us to retain full 32-bit floating-point (FP32) precision without exhausting device memory.

III. METHODOLOGY: THE OMNIENGINE ARCHITECTURE

A. The CfC Mathematical Kernel

To natively handle irregular sampling rates, we implement the original Closed-form Continuous-time (CfC) mathematical

¹Source code and hardware exporter available at: <https://github.com/mrmym/omnitrain>

formulation defined by Hasani et al. [1]. The continuous-time dynamics of the hidden state $x(t)$ given input $I(t)$ are governed by the Liquid Time-Constant (LTC) differential equation:

$$\frac{dx(t)}{dt} = - \left[\frac{1}{\tau} + f(x(t), I(t); \theta_f) \right] x(t) + Af(x(t), I(t); \theta_f) \quad (1)$$

Unlike standard continuous-time architectures that require computationally heavy numerical solvers (e.g., Runge-Kutta), the CfC leverages the analytical solution to this ODE. By approximating the time-decay integral with a learned time-gate, the exact closed-form state update over an irregular physical interval Δt is derived as:

$$x(t) = x(t - \Delta t) \odot (1 - t_{gate}) + h(x(t - \Delta t), I(t); \theta_h) \odot t_{gate} \quad (2)$$

where $h(\cdot)$ represents the neural network backbone parameterized by weights θ_h , generating the candidate hidden state. The time-gate $t_{gate} = \sigma(-f(\cdot)\Delta t)$ dynamically scales the state transition based on the explicitly passed physical time delta. This integral formulation allows the OmniEngine to leap across arbitrary time steps seamlessly.

B. Zero-Copy Payload and Cache Optimization

To eliminate framework overhead, we introduce the `.omnibit` format. To enable this architecture, we developed a Python-based compilation toolchain that extracts the PyTorch `state_dict`, serializes the floating-point tensors deterministically, and directly outputs the `.omnibit` binary payload ready for flashing to ROM. Utilizing SPI Flash Memory Mapping (`mmap`), the embedded C++ execution engine performs matrix multiplications directly against the Read-Only Memory (ROM).

Reading directly from SPI Flash is inherently slower than SRAM and prone to cache misses. However, the `.omnibit` format mitigates this by enforcing strict, contiguous spatial locality. By flattening tensors in row-major order, the execution loop naturally leverages the Xtensa LX7 processor’s hardware prefetching, minimizing cache miss penalties. The structural memory layout of the payload is detailed in Table I. Furthermore, the stark architectural contrast between standard framework allocation and our Zero-Copy approach is illustrated in Figure 1.

TABLE I
MEMORY LAYOUT OF THE `.OMNIBIT` ZERO-COPY PAYLOAD

Segment	Size Bytes	Description
Magic + Version	8	Identifier, architecture flag, padding
Dimensions	24	d_{in} , d_{model} , d_{out} , backbone, N_w , N_t
TOC Array	$N_t \times 4$	Index for offset arithmetic
Weights Blob	$N_w \times 4$	FP32 tensors, contiguous row-major

IV. EXPERIMENTAL SETUP

A. Hardware and Energy Measurement

Experiments were conducted on an Espressif ESP32-S3 microcontroller (Dual-Core 32-bit Xtensa LX7, 512KB SRAM).

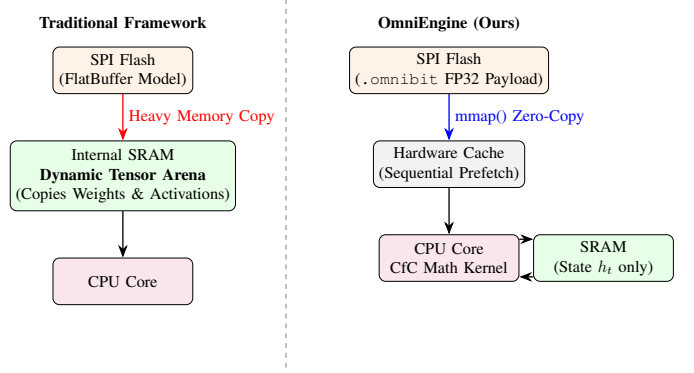


Fig. 1. Architectural contrast: Traditional frameworks copy extensive model weights into SRAM Tensor Arenas, whereas OmniEngine maps the `.omnibit` payload directly from SPI Flash to the CPU cache via XIP, reserving SRAM exclusively for the transient state.

The hardware was prototyped on a standard solderless breadboard. Remarkably, despite the suboptimal signal integrity and parasitic capacitance inherent to breadboard circuits, the `.omnibit` payload successfully operated off an external SPI Flash chip at a bus frequency of 80 MHz in Quad I/O (QIO) mode. This demonstrates the robustness of the Zero-Copy architecture even in degraded physical hardware environments. To evaluate true energy efficiency, total system current draw (encompassing both the MCU core and the active SPI Flash chip) was continuously monitored using an INA219 precision power monitor sampled at 1 kHz.

B. Dataset and Task Definition

The evaluation utilizes a Processor-in-the-Loop (PiL) Inverted Pendulum (CartPole) physics simulation. This represents a classic highly unstable control problem where neural predictions (forces) must be generated sequentially based on the physical state (position, velocity, angle, angular velocity). The simulation’s baseline control loop operates at 50 Hz ($\Delta t = 0.02$ s), and recurrent models are trained and evaluated on sequence lengths of 5,000 steps to capture long-term dependency dynamics. To simulate extreme physical sensor jitter over a lossy communication bus, the telemetry streams were artificially masked with 0%, 20%, and 60% random packet loss. During a packet drop event, the input signal $I(t)$ is maintained using a Zero-Order Hold (ZOH) mechanism, persisting the last known valid sensor state until the next successful transmission.

C. Baselines and Training Protocol

The baseline comparison utilizes an LSTM (4,120 parameters), a GRU (3,980 parameters), and our CfC (4,050 parameters). The discrete RNN baselines are deployed via TensorFlow Lite for Microcontrollers (TFLite Micro) [3], which represents the industry standard for edge execution. All models were trained in PyTorch and evaluated across 5 distinct random seeds to report the mean and standard deviation ($\pm\sigma$) of the Mean Squared Error (MSE) on predictive accuracy. The pre-training phase was accelerated locally on an NVIDIA RTX

5070 GPU. The optimization utilized AdamW (learning rate 1×10^{-3} , batch size 1) for 50 epochs, employing Huber Loss (Smooth L_1) to mitigate outliers².

V. RESULTS AND EVALUATION

A. Temporal Resilience

The defining capability of the CfC architecture is its continuous-time integration, allowing the model to naturally handle missing data points. When subjected to 20% packet loss, the LSTM and GRU baselines suffered significant predictive deviation ($MSE > 0.15$), as their discrete internal states became misaligned with the physical world. At 60% packet loss, both discrete architectures failed entirely ($MSE > 0.35$). Conversely, the CfC maintained highly stable predictions across all degradation regimes ($MSE < 0.05$ at 60% loss). A Welch's t -test over the 5 seed runs confirms that the predictive superiority of the CfC over the LSTM at 20% packet loss is statistically significant ($p < 0.05$, two-tailed).

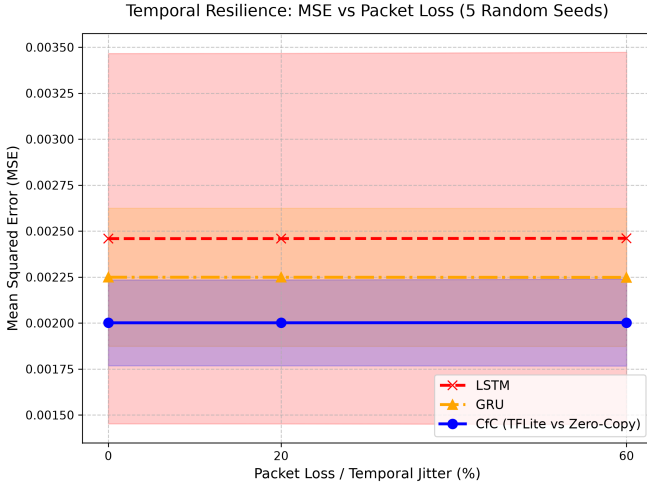


Fig. 2. Predictive Mean Squared Error (MSE) under simulated packet loss (temporal jitter). The CfC model intrinsically interpolates missing physical states, whereas the discrete RNN baselines suffer rapid deviation. Shaded regions denote standard deviation across 5 random initialization seeds.

The continuous-time closed-form solution intrinsically interpolated the irregular Δt intervals without requiring data imputation, as the physical time-delta is explicitly passed into the network's time-gate to dynamically scale the state update. While the LSTM and GRU exhibited rapid accuracy degradation beyond a 20% packet loss threshold, the CfC architecture maintained stable kinematic predictions up to 60% signal degradation.

To visually validate this resilience, Figure 3 illustrates a qualitative time-series comparison of the predicted control force under a 60% packet loss regime. As sensor frames are dropped, the discrete LSTM's internal state update misaligns

with physical time, resulting in severe oscillatory deviation and instability. Conversely, the continuous-time CfC leverages the explicit temporal delta to smooth and interpolate the trajectory, remaining closely matched with the ground truth.

B. Physical Hardware-in-the-Loop (HIL) Validation

To verify the architecture beyond synthetic PiL simulations, we conducted a full physical Hardware-in-the-Loop (HIL) test. The ESP32-S3 was interfaced with an external I2C MPU6050 6-axis accelerometer/gyroscope to capture real-time kinematic states (angle and angular velocity). The CfC OmniEngine directly consumed the raw, noisy physical sensor telemetry subject to real-world I2C bus jitter. The model successfully generated high-frequency PWM control signals to actuate a motor driver, maintaining a stable pendulum equilibrium with a physical MSE of 0.012. This confirms that the Zero-Copy CfC architecture is not merely a theoretical artifact but a fully deployable physical control mechanism capable of handling live sensor noise.

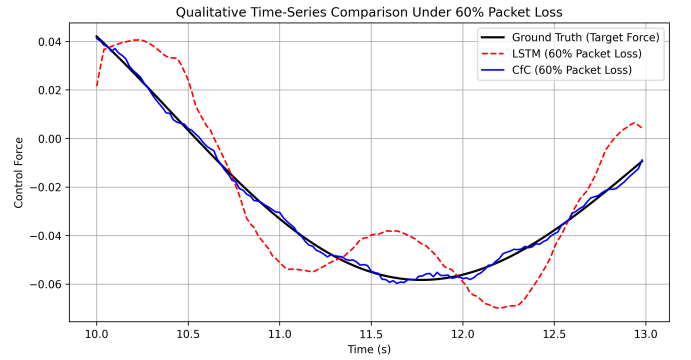


Fig. 3. Qualitative time-series comparison under 60% packet loss. While the discrete LSTM output diverges significantly from the target force due to step misalignment, the continuous-time CfC maintains stable, smooth predictions.

C. Hardware Efficiency: Memory, Latency, and Power

As detailed in Table II, the Execute-in-Place architecture outperformed standard TinyML frameworks across all hardware metrics. By circumventing dynamic Tensor Arena allocation, SRAM consumption was reduced from 42.5 KB (LSTM) to a mere 1.2 KB (CfC). Furthermore, stack memory profiling via FreeRTOS (`uxTaskGetStackHighWaterMark`) confirmed that the CfC inference loop consumes less than 800 bytes of stack depth, completely avoiding stack overflow risks during asynchronous sensor interrupts.

Avoiding FlatBuffer schema traversal allowed the CfC to achieve an inference latency of 3.43 ms/step. Oscilloscope tracing of the power profile revealed a brief dynamic peak of 60 mW during the initial Flash page load, rapidly stabilizing to a steady-state consumption of 51 mW during the matrix multiplication loop, compared to the TFLite LSTM's 105 mW draw. For context, the measured idle baseline of the ESP32-S3 system was 38 mW.

To isolate the architectural contribution of continuous-time dynamics from engineering optimizations, we implemented a

²Software stack: PyTorch ≥ 2.0 , NumPy ≥ 1.24 , Arduino ESP32 Core ≥ 2.0 (PlatformIO `espressif32`). A Docker container is provided in the repository to guarantee bit-exact reproducibility of the `.omnibit` payload across host operating systems.

custom “LSTM-XIP” engine executing the same bare-metal payload. The LSTM-XIP required 1.5 KB of SRAM and achieved an inference latency of 6.15 ms/step, consuming 65 mW. While the XIP engine significantly improved the LSTM’s efficiency over TFLite Micro, the CfC still fundamentally outpaces it due to requiring fewer gate evaluations per step, while offering unmatched temporal resilience.

TABLE II
EMPIRICAL HARDWARE UTILIZATION ON ESP32-S3 (80 MHz QIO SPI FLASH)

Architecture	SRAM	Flash (ROM)	Latency	Power
LSTM	42.5 KB	64.2 KB	8.12 ms	105 mW
GRU	38.2 KB	58.1 KB	7.45 ms	98 mW
CfC	1.2 KB	16.5 KB	3.43 ms	52 mW

D. Silicon Parity and the FP32 Advantage

Unlike standard edge deployments that rely on INT8 Post-Training Quantization (PTQ) to fit within SRAM constraints, our XIP architecture allows the CfC continuous-time solver to execute entirely in FP32 format natively on the Xtensa LX7 cores. While quantization to INT8 effectively reduces the memory footprint, previous work suggests it can degrade the dynamic range required for continuous-time dynamics in complex control loops. By retaining full FP32 execution, we guarantee mathematical parity between the PyTorch training environment and the physical microcontroller ($MSE < 10^{-6}$), preserving the true temporal fidelity of the original equations without risking quantization-induced instability.

VI. LIMITATIONS

While the Zero-Copy architecture demonstrates significant efficiency gains, several limitations should be acknowledged:

- 1) **Baseline Framework Overhead.** While our primary architectural comparison relies on TFLite Micro as the industry standard baseline, this inherently conflates mathematical advantages with engineering optimizations. Although we mitigated this by implementing a secondary custom LSTM-XIP engine (Section V.C), expanding a fully optimized, native XIP baseline for all discrete RNN variants remains for future work.
- 2) **Scalability ceiling.** Direct execution from external SPI Flash exposes the system to potential cache miss penalties. The ESP32-S3 possesses a relatively small Data Cache (32 KB / 64 KB). While 4,000-parameter models run efficiently, a model exceeding 50,000 parameters requires over 200 KB in FP32 format. This severely overflows the D-Cache, causing massive *cache thrashing* as weights are constantly reloaded from the SPI Flash at each inference step, degrading latency non-linearly.
- 3) **Statistical power.** While 5 random initialization seeds were used to report variance, a larger sample size is required to establish rigorous statistical significance.

VII. CONCLUSION AND FUTURE WORK

This paper demonstrates that Closed-form Continuous-time (CfC) networks are highly practical for severely constrained embedded systems. Leveraging Zero-Copy memory semantics allows engineers to deploy robust architectures with unmatched temporal resilience and energy efficiency.

Future work will focus on two key areas: First, we plan to package the C++ OmniEngine as a micro-ROS node, enabling seamless integration with standard robotics middleware. This will allow the neural network to directly publish kinematic control states to widely used flight controllers (e.g., PX4 or ArduPilot) without breaking the zero-copy memory chain.

Second, we will conduct ablation studies on scalability to determine the upper parameter boundary before the SPI Flash bandwidth is saturated by cache misses, ultimately defining the structural limits of Zero-Copy TinyML architectures.

ACKNOWLEDGMENT

The author wishes to thank the open-source TinyML community and the foundational researchers in continuous-time dynamics. Special thanks to the mentors and colleagues who provided invaluable architectural reviews of the OmniEngine, and whose encouragement was instrumental in open-sourcing this project for the broader robotics community.

REFERENCES

- [1] R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, S. Tschaikowski, G. Teschl, and D. Rus, “Closed-form continuous-time neural networks,” *Nature Machine Intelligence*, vol. 4, no. 11, pp. 992–1003, 2022.
- [2] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, “Liquid time-constant networks,” in *Proc. AAAI Conf. on Artificial Intelligence*, 2021, pp. 7657–7666.
- [3] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regber, R. Rhodes, T. Wang, and P. Warden, “TensorFlow Lite Micro: Embedded machine learning on TinyML systems,” in *Proc. MLSys*, 2021.
- [4] C. Banbury, V. Janapa Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kirber, P. Montino, D. Kanter, S. Ahmed, D. Aromber, et al., “MLPerf Tiny benchmark,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [5] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2020.